

Chapter 4

Threads and Concurrency

同時執行、同時進行

Chih-Yu Lin

National Taiwan Ocean University

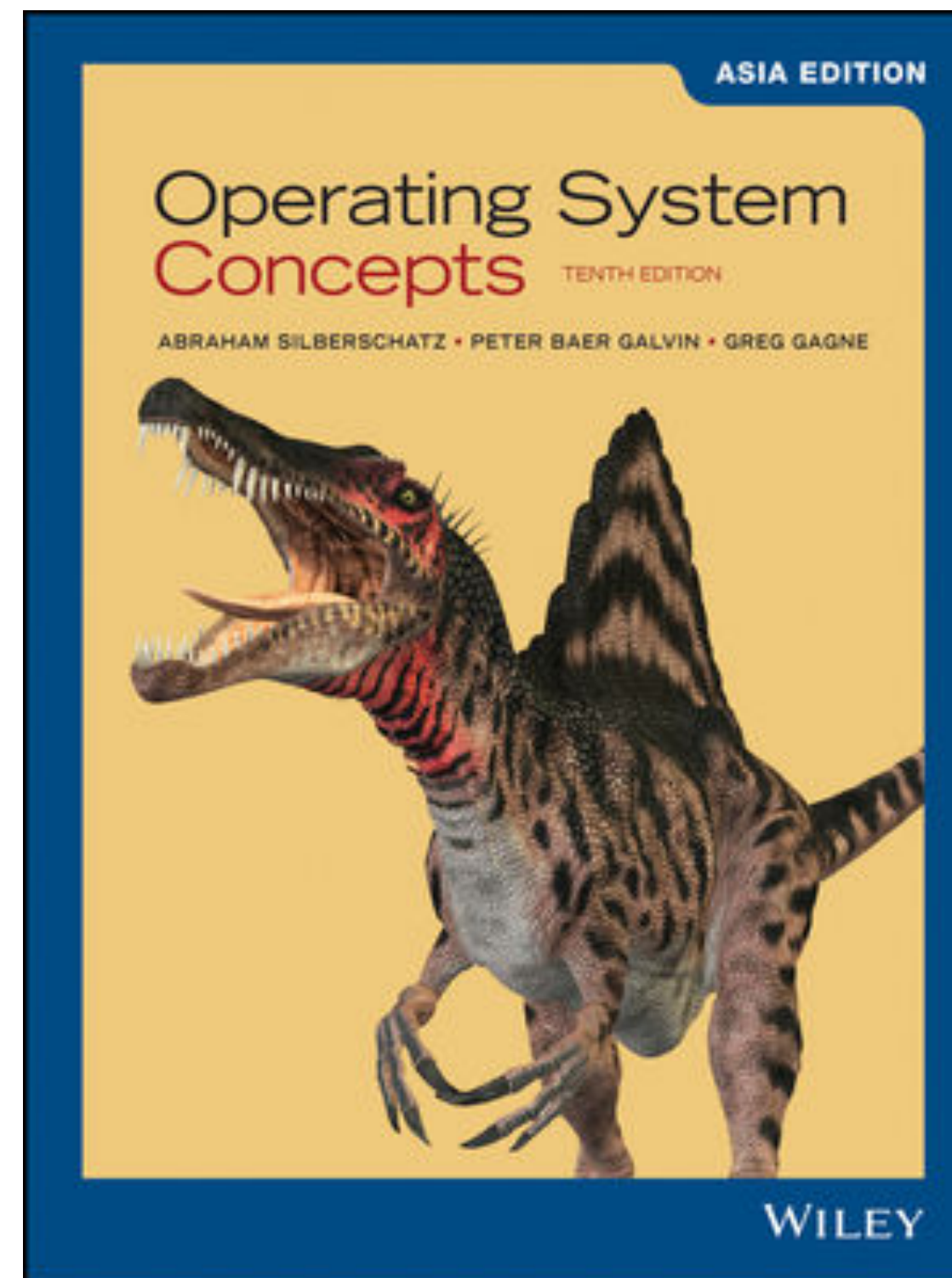
Concurrency

可能發生在 MultiThreads，也可能
發生在 Multi Process

Updated date: 2025/7/8

Textbook

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, "Operating System Concepts, Asia Edition," 10/e, ISBN: 978-1-119-58616-6, Wiley



Outline

- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues

Outline

- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues

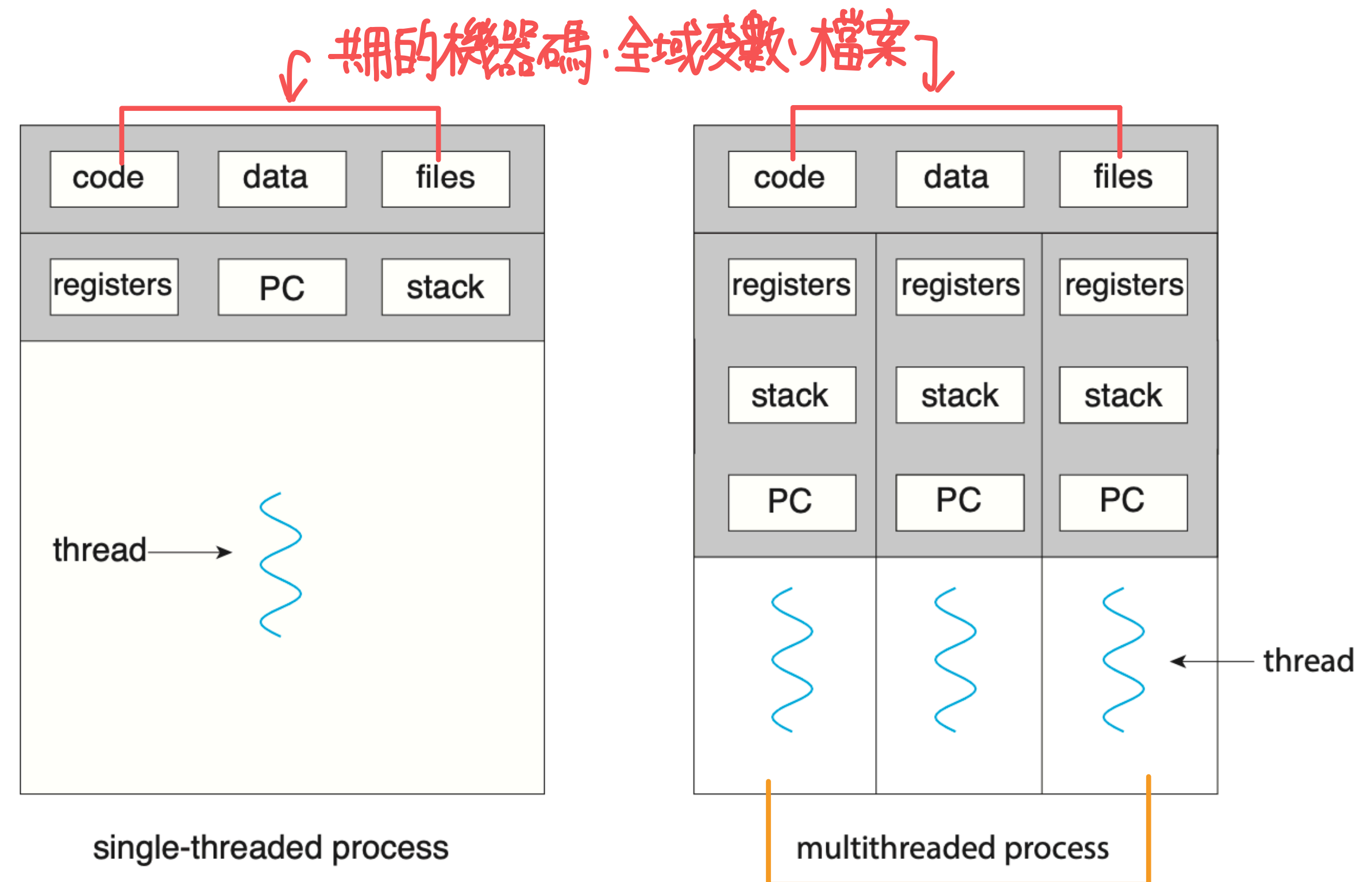
Thread

process 和 thread 的差別

Process:

- fork() System Call
- Child Process 和 Parent 有不同記憶體空間
- 通常 Child 跟 Parent 執行的東西是不一樣的

- Light-weight (又稱 light weight Process)
- Comprises a thread ID, a program counter (PC), a register set, and a stack
- Shares with other threads belong to the same process its code section, data section, and other operating-system resources, such as open files and signals



每個 Thread 有配 獨立的 Program Counter、Stack、Reg

ProcessThread.c

共用全域!

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <pthread.h>
5  #include <sys/wait.h>
6
7  // 宣告全域變數
8  int global_var = 0;
9
10 // Thread 函數
11 void* thread_function(void* arg) {
12     int thread_id = *(int*)arg;
13
14     printf("Thread %d: 初始 global_var = %d\n", thread_id, global_var);
15
16     // 修改全域變數
17     global_var += 10;
18     printf("Thread %d: 修改後 global_var = %d\n", thread_id, global_var);
19
20     // 等待一秒讓其他 thread 也有機會執行
21     sleep(1);
22
23     printf("Thread %d: 最終 global_var = %d\n", thread_id, global_var);
24
25     return NULL;
26 }
```

--- Thread 測試 (共用記憶體空間) ---

Thread 1: 初始 global_var = 0

Thread 1: 修改後 global_var = 10

Thread 2: 初始 global_var = 10

Thread 2: 修改後 global_var = 20

Thread 3: 初始 global_var = 20

Thread 3: 修改後 global_var = 30

Thread 1: 最終 global_var = 30

Thread 2: 最終 global_var = 30

Thread 3: 最終 global_var = 30

主程序中的 global_var (thread 測試後): 30

獨立的資料 (mem 內有 4 份全域變數)

--- Process 測試 (獨立記憶體空間) ---

Process 1: 初始 global_var = 0

Process 1: 修改後 global_var = 10

Process 2: 初始 global_var = 0

Process 2: 修改後 global_var = 10

Process 3: 初始 global_var = 0

Process 3: 修改後 global_var = 10

Process 1: 最終 global_var = 10

Process 2: 最終 global_var = 10

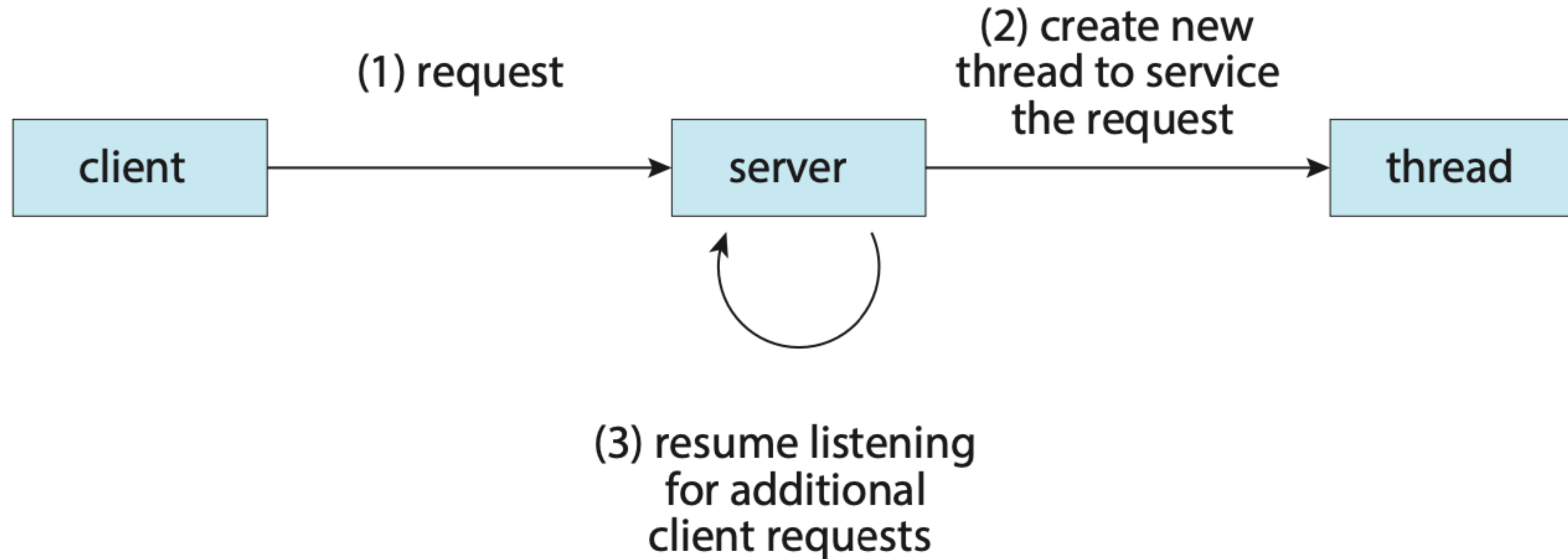
Process 3: 最終 global_var = 10

主程序中的 global_var (process 測試後): 0

Motivation

- Most modern applications are multithreaded
- Threads run within application (一个App内可能有多个Thread)
- Multiple tasks with the application can be implemented by separate threads
 - Update display (UI / UX)
 - Fetch data (下載資料)
 - Spell checking
 - Answer a network request (e.g., Android)
- Process creation is heavy-weight while thread creation is light-weight
- Can simplify code, increase efficiency
- Kernels are generally multithreaded

Multithreaded Server Architecture



Benefits

└ UI 做為 Main Threads, 保證其他 task 不影響使用者操作

- **Responsiveness** – may allow **continued execution** if part of process is blocked, especially important for **user interfaces**
- **Resource Sharing** – threads share resources of process, easier than shared memory or message passing
- **Economy** – cheaper than process creation, thread switching lower overhead than context switching
- **Scalability** – can take advantage of **multiprocessor architectures**

└ 可以使用多個 Core

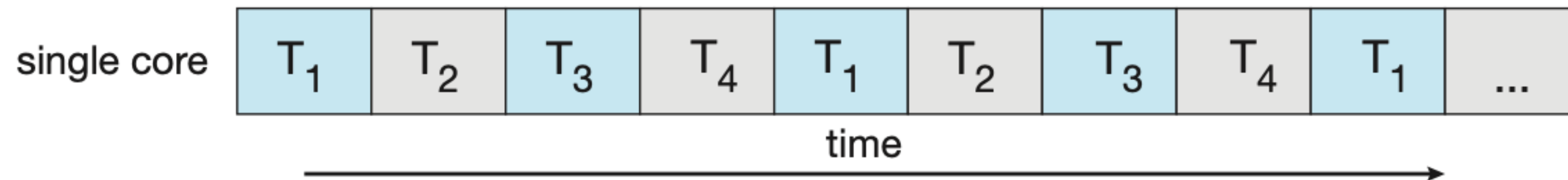
Outline

- Overview
- Multicore Programming Thread & Process 都可以做 Multi Core Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues

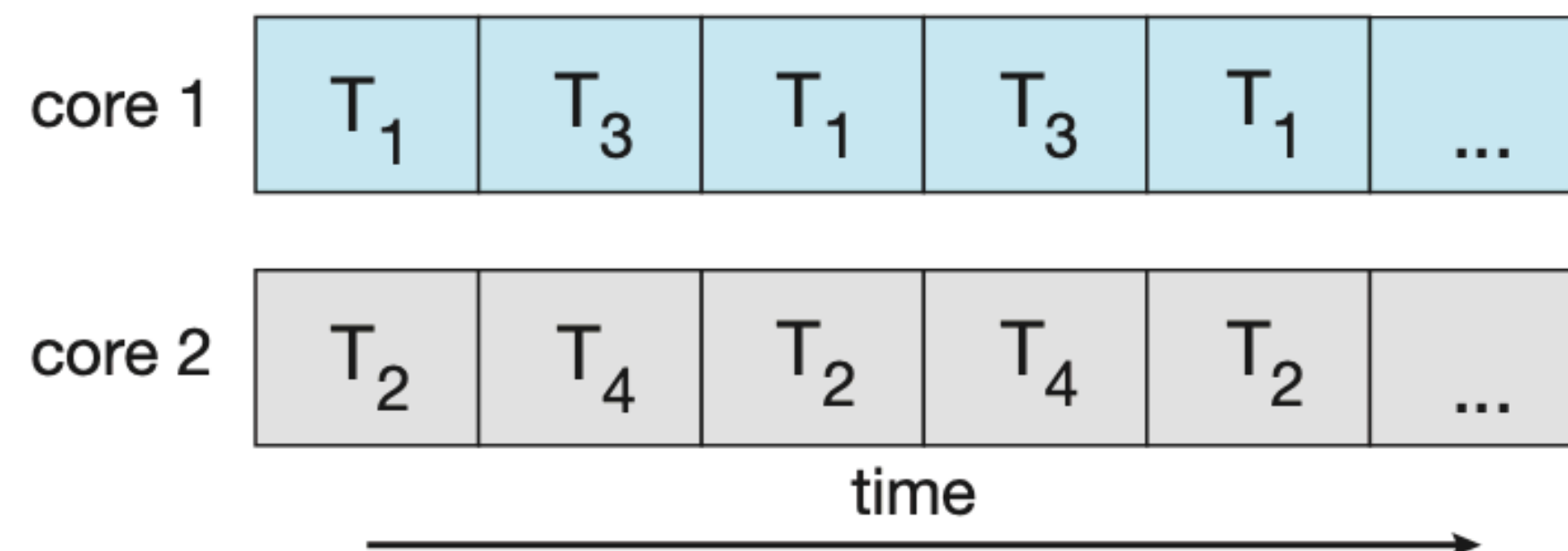
Concurrency vs. Parallelism

近似 Time Sharing

- **Concurrent execution** on a single-core system
(類似 Time sharing)



- **Parallel execution** on a multicore system
物理上的同時 (平行執行)



Multicore Programming

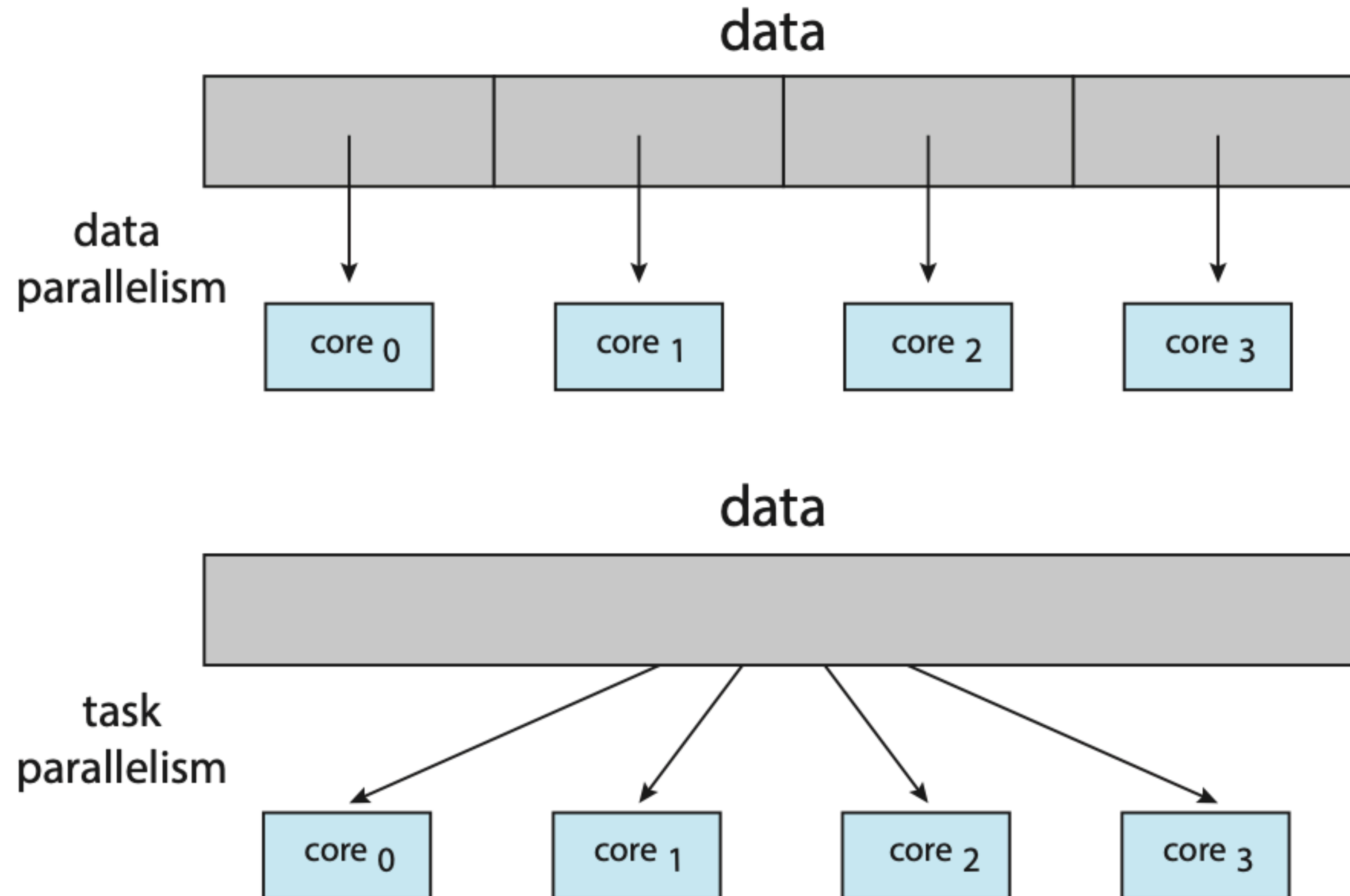
- Multicore or multiprocessor systems putting pressure on programmers, challenges include:
 - Dividing activities (Identifying tasks)
 - Ideally, tasks are independent of one another and thus can run in parallel on individual cores → 理想上，每个 Thread 所執行的任務是不同的
 - Balance (equal)
 - Data splitting
 - Data dependency (兩個 Threads 同時存取一个变數 → 產生同步問題！)
 - When one task depends on data from another, programmers must ensure that the execution of the tasks is synchronized (Chapter 6)
 - Testing and debugging

Multicore Programming

- Types of parallelism 類似作業1找質數
- **Data parallelism** - distributes subsets of the same data across multiple cores, same operation on each
- Example: sum the elements $[0] \dots [N-1]$
 - Thread A (core 0): $[0] \dots [N/2-1]$
 - Thread B (core 1): $[N/2] \dots [N-1]$
- **Task parallelism** - distributing tasks (threads) across cores, each thread performing unique operation 有2个不同的事情要同時處理
- Image Processing
 - Thread A: Resize the image to standard dimensions
 - Thread B: Adjust color balance for better visual appearance

会考

Data and Task Parallelism



Amdahl's Law

阿姆達爾定律

- Identifies performance gains from adding additional cores to an application that has both **serial** and **parallel** components

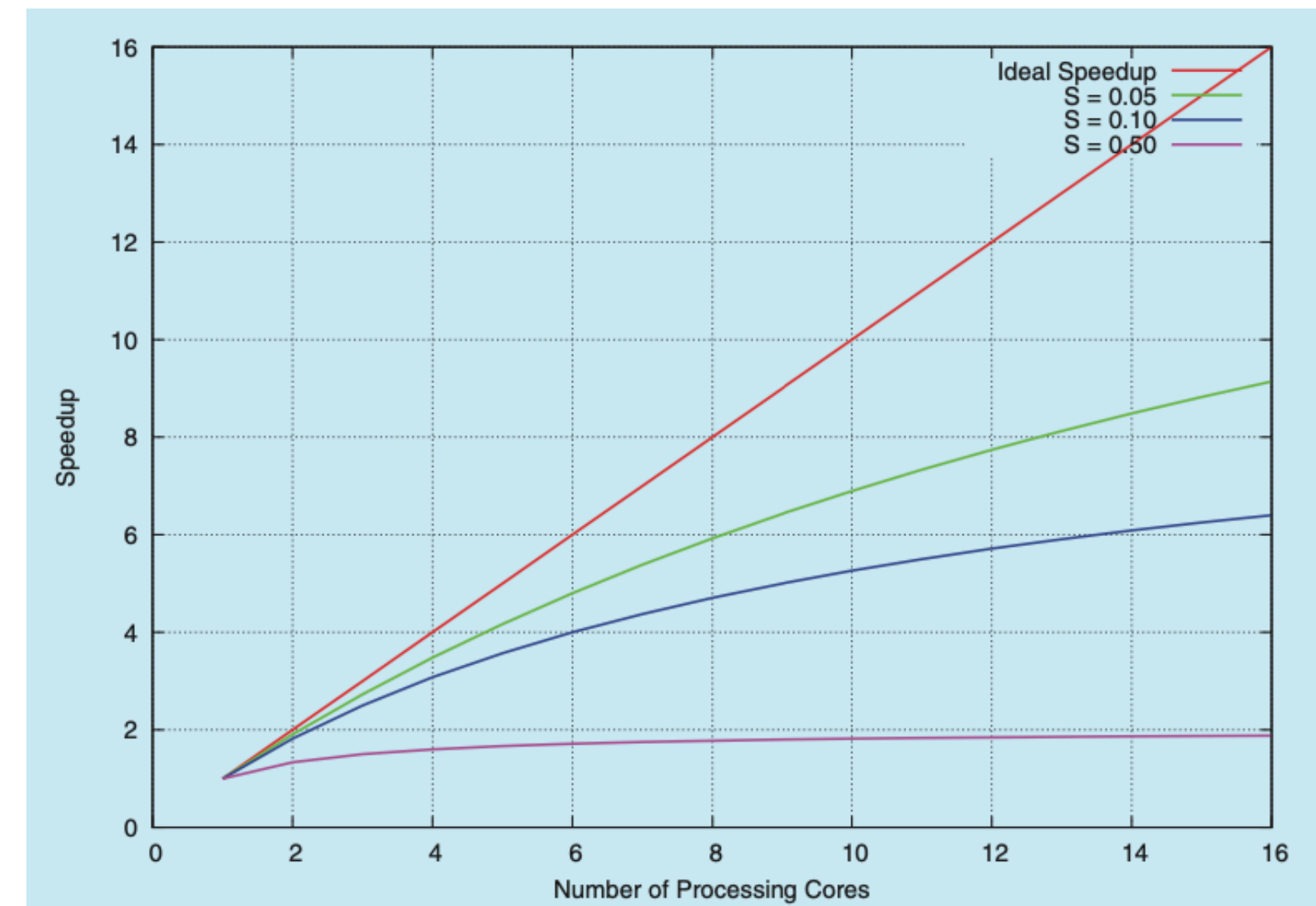
- S is serial portion

- N processing cores

會考

$$\text{speedup} \leq \frac{1}{S + \frac{(1-S)}{N}}$$

$$\frac{1}{0.25 + \frac{(1-0.25)}{2}} = 1.6$$



- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As N approaches infinity, speedup approaches 1/S

一個程式若能平行化的比例越高, 用 Multi Core Programming 帶來的效能改善越多

Examples

- $1 + 2 + 3 + \dots + 1000$ (Easy)
- Fibonacci Sequence (Hard)
 - $F_n = F_{n-1} + F_{n-2}$
- Loop Interchange / Loop Reordering

Outline

- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues

User Threads and Kernel Threads

由程式設計師撰寫程式時模擬 Multi Program Counter

- User threads - management done by user-level threads library (e.g., Java Thread)
- Kernel threads - Supported by the operating system (由 OS 提供一真正的 Thread)
- A relationship must exist between user threads and kernel threads
 - Many-to-One Model (有問題)
 - One-to-One Model
 - Many-to-Many Model

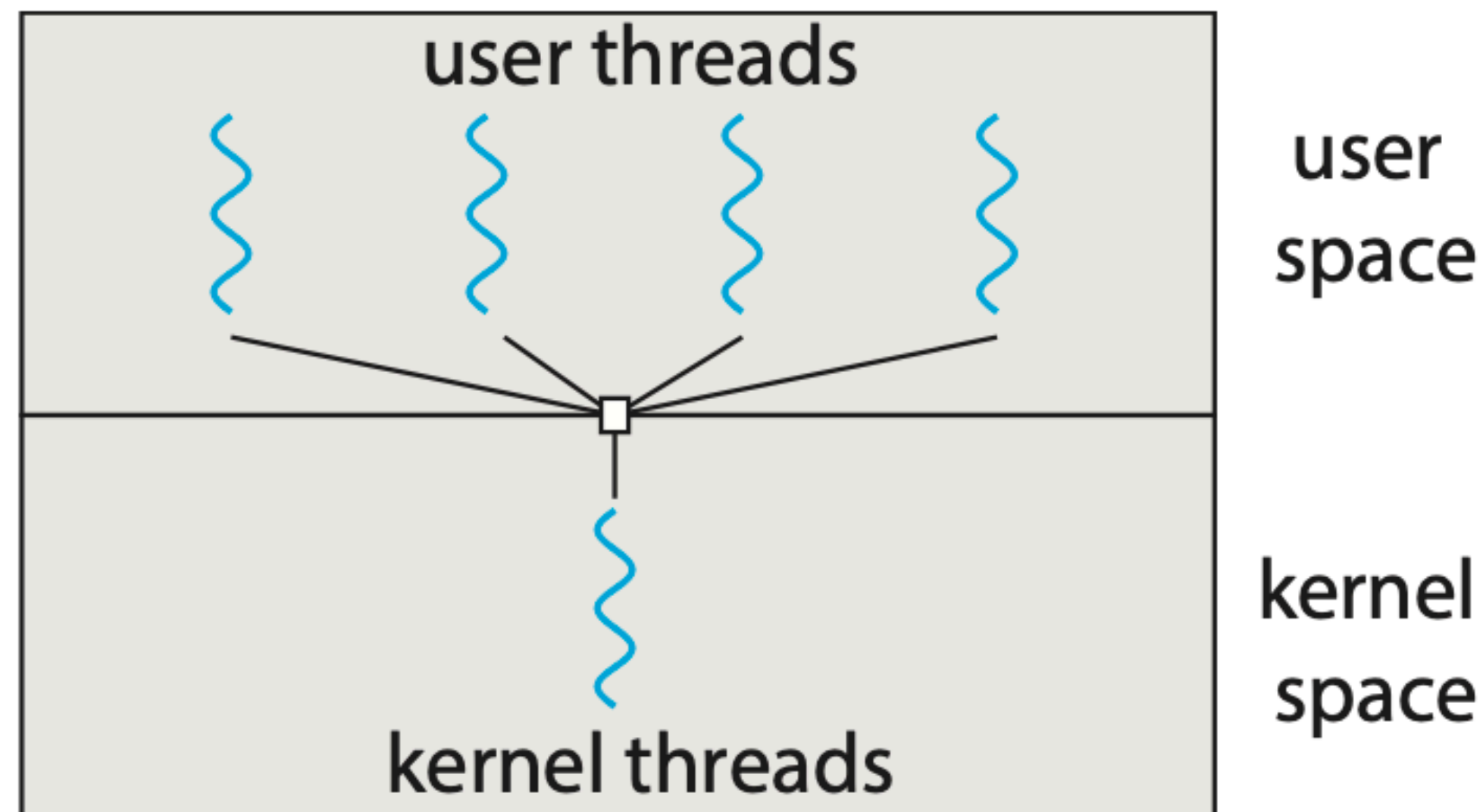
Many-to-One Model

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time
- Few systems currently use this model

User 端有多个 Thread

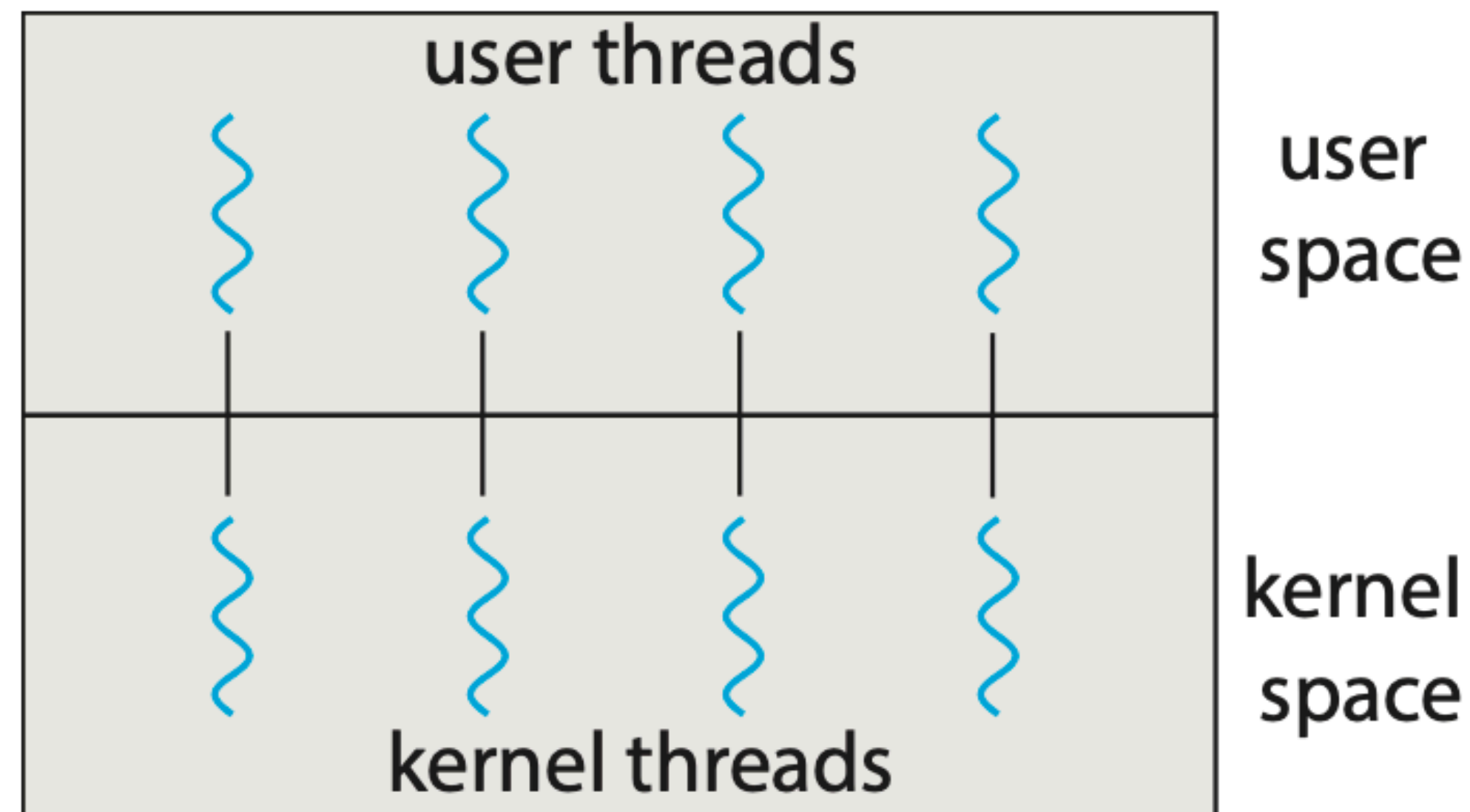
OS 端只提供 Single Thread

模拟出来的, 不是真正的“同时”



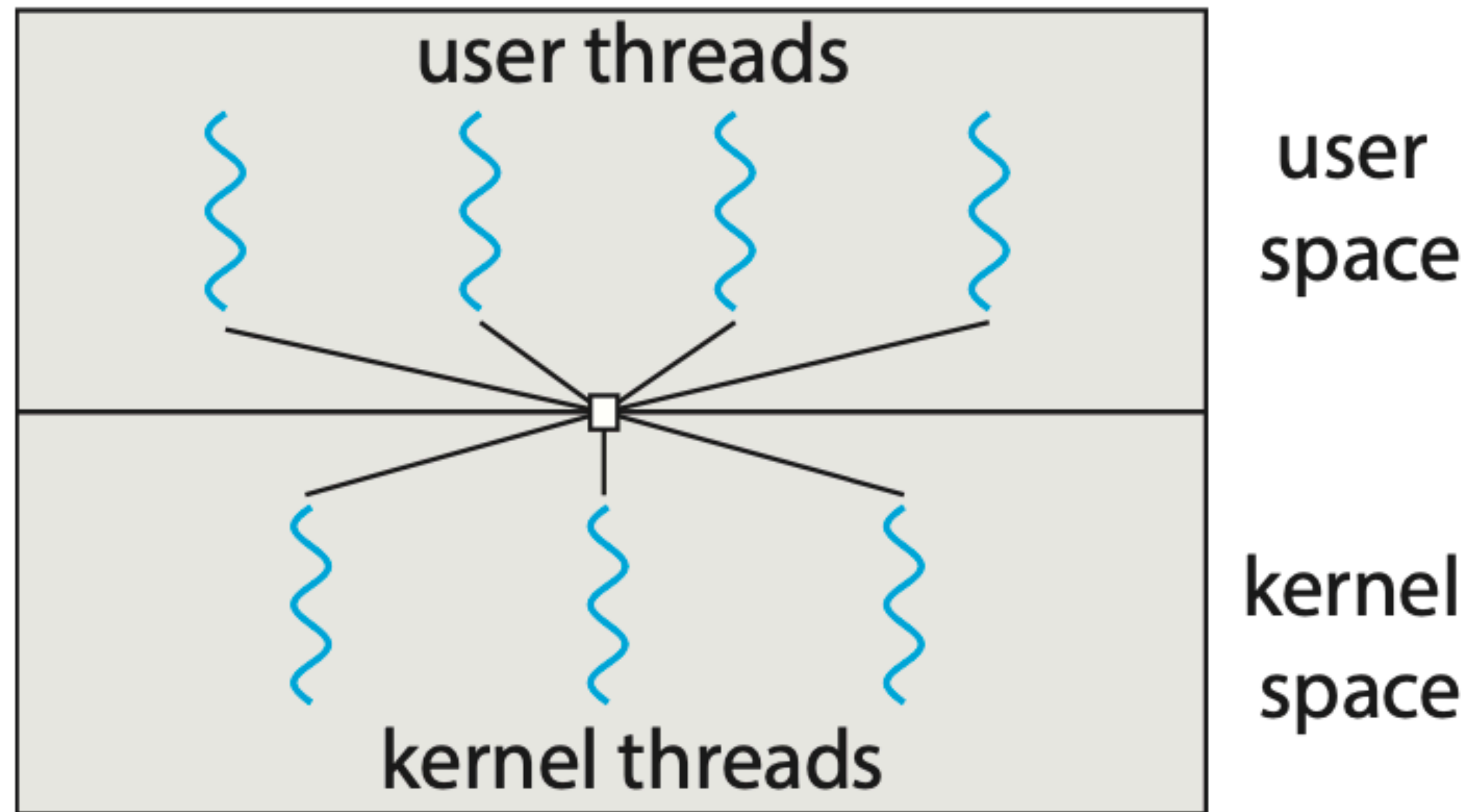
One-to-One Model

- Creating a user-level thread creates a kernel thread 一个 user thread 就对应一个 kernel thread
- The only drawback to this model is that creating a user thread requires creating the corresponding kernel thread, and a large number of kernel threads may burden the performance of a system → 如果有大量的 Thread 時，会导致 System 效能的負擔
- Linux, along with the family of Windows operating systems, implement the one-to-one model. (Textbook) → 現今 OS 多採用 one-to-one



Many-to-Many Model

- Multiplexes many user-level threads to a **smaller** or equal number of kernel threads

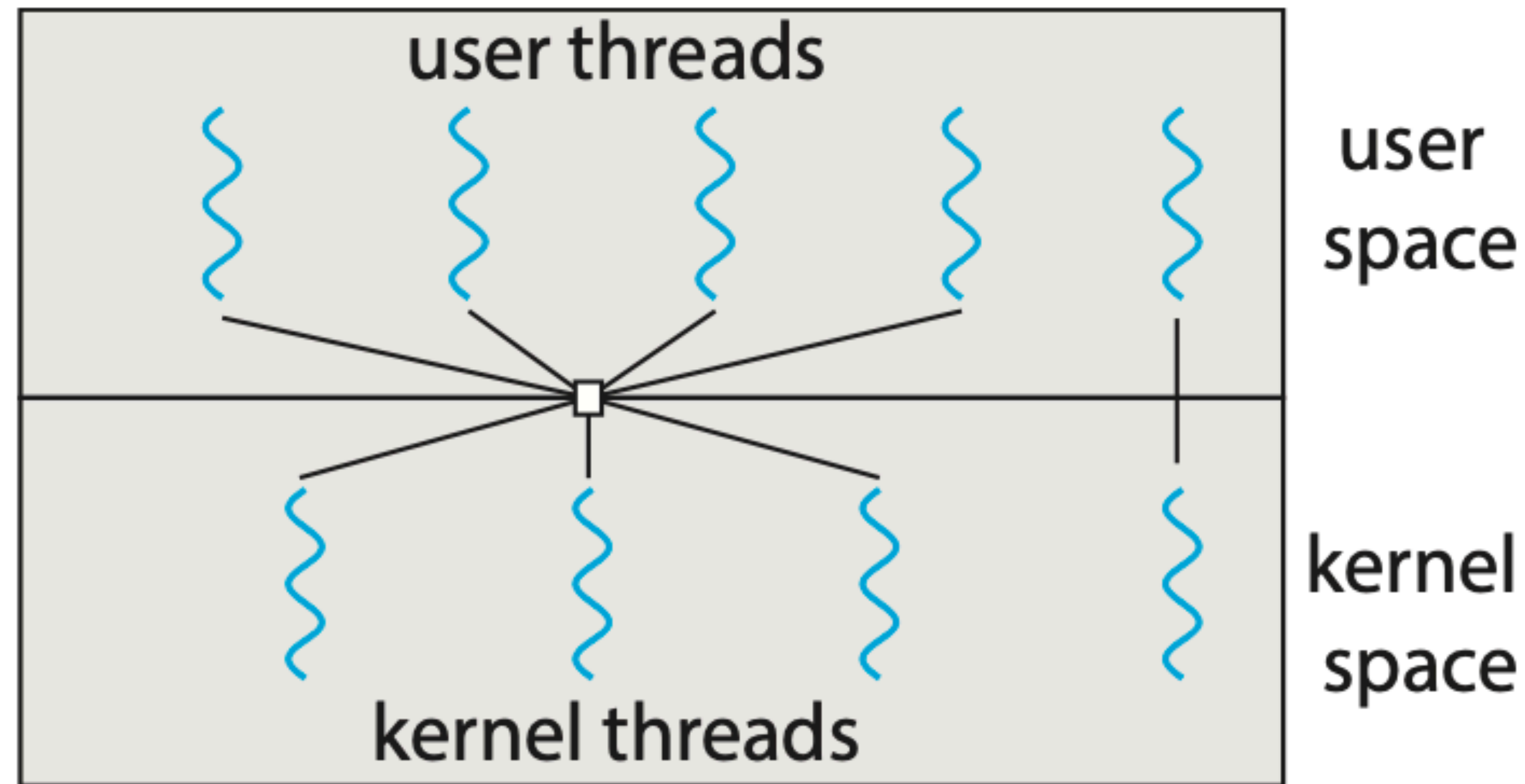


kernel Threads < User Threads

Two-Level Model

重要的 Thread 採用 one-to-one
剩下的用 Many-to-Many

- Multiplexes many user-level threads to a **smaller** or equal number of kernel threads but **also allows a user-level thread to be bound to a kernel thread**



Outline

- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues

Pthreads

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }
}
```

```
/* get the default attributes */
pthread_attr_init(&attr);
/* create the thread */
pthread_create(&tid, &attr, runner, argv[1]);
/* wait for the thread to exit */
pthread_join(tid, NULL);

printf("sum = %d\n", sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```

Pthreads

- SleepSortThread.c

```
76      // Create thread
77      if (pthread_create(&threads[i-1], NULL, sleep_sort_thread, data) != 0) {
78          perror("Thread creation failed");
79          free(data);
80          free(threads);
81          pthread_mutex_destroy(&print_mutex);
82          return 1;
83      }
84  }
85
86  // Wait for all threads to complete
87  for (int i = 0; i < num_threads; i++) {
88      pthread_join(threads[i], NULL);
89  }
```

Java Threads

- Java threads are managed by the JVM
- Typically implemented using the threads model provided by underlying OS
- Java threads may be created by
 - Extending **Thread class**
 - Implementing the **Runnable interface**

```
class Task implements Runnable
{
    public void run() {
        System.out.println("I am a thread.");
    }
}

Thread worker = new Thread(new Task());
worker.start();
```


Java Threads

```
5 public static void main(String[] args) {
6     RaceConditionExample example = new RaceConditionExample();
7     example.main();
8 }
9
10 void main() {
11
12     this.x = 3;
13
14     AddThread t1 = new AddThread(this, 1);
15     AddThread t2 = new AddThread(this, 2);
16
17     t1.start();
18     t2.start();
19     try {
20         Thread.sleep(6000);
21     } catch (InterruptedException e) {
22     }
23     System.out.println(x);
24 }
```

```
26 class AddThread extends Thread {
27
28     int addValue = 0;
29     RaceConditionExample example;
30
31     public AddThread(RaceConditionExample example, int addValue) {
32         this.addValue = addValue;
33         this.example = example;
34     }
35
36     @Override
37     public void run() {
38         example.addX(addValue);
39     }
40 }
```

Further Reading: Java **Executor** Framework:

<https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Executor.html>

Outline

- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues

Implicit Threading

不用特別指定說要開一個新的 Thread，而是由 library 來管理 Thread

- Growing in popularity as numbers of threads increase, program correctness more difficult with **explicit threading** (programmers manually create, manage, and synchronize threads throughout the application lifecycle) → 要求 Programmer 手動管理、創造 Thread)
- Creation and management of threads done by **compilers** and **run-time libraries** rather than programmers → 由 Compiler & library 來建立、管理 Thread
- require programmers to identify **tasks** - not **threads** - that can run in parallel
- Three methods explored
 - Thread Pools (Java)
 - OpenMP (C Language)

Thread Pools

- Create a number of threads in a pool where they await work
- Advantages:
 - Usually slightly faster to service a request
 - Allows the number of threads in the application(s) to be bound to the size of the pool
 - Separating task to be performed from mechanics of creating task allows different strategies for running task

Java Thread Pools

1. Single thread executor—`newSingleThreadExecutor()`—creates a pool of size 1.
2. Fixed thread executor—`newFixedThreadPool(int size)`—creates a thread pool with a specified number of threads.
3. Cached thread executor—`newCachedThreadPool()`—creates an unbounded thread pool, reusing threads in many instances.

Java Thread Pools

```
import java.util.concurrent.*;

public class ThreadPoolExample
{
    public static void main(String[] args) {
        int numTasks = Integer.parseInt(args[0].trim());

        /* Create the thread pool */
        ExecutorService pool = Executors.newCachedThreadPool();

        /* Run each task using a thread in the pool */
        for (int i = 0; i < numTasks; i++)
            pool.execute(new Task());

        /* Shut down the pool once all threads have completed */
        pool.shutdown();
    }
}
```


OpenMP

編譯器 指導符 (假指令)

- OpenMP is a set of **compiler directives** as well as an API for programs written in C and C++

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    /* sequential code */

    #pragma omp parallel ← 要求編譯器对此平行處理
    {
        printf("I am a parallel region.");
    }

    /* sequential code */

    return 0;
}
```

Outline

- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues

Threading Issues

- Some of the issues to consider in designing multi-threaded programs
 - The fork() and exec() System Calls
 - Signal Handling
 - Thread Cancellation
 - Thread-Local Storage (TLS)
 - Scheduler Activations

The fork() and exec() System Calls

程式 Call fork 時, 子process 是 single-thread 还是 multi-thread?

- If one thread in a program calls fork(), does the new process duplicate all threads, or is the new process **single-threaded**?
- Reference: <https://pubs.opengroup.org/onlinepubs/009696799/functions/fork.html>
- Some UNIX systems have chosen to have two versions of fork()
- The exec() system call typically works as normal
 - Replace the entire process - including all threads

Signal Handling

- A **signal** is used to notify a process that a particular event has occurred
 - Examples
 - Common Termination Signals
 - SIGTERM (15) - Termination signal, requests normal process termination, can be caught and ignored
 - SIGKILL (9) - Forced termination signal, cannot be caught or ignored, immediately terminates the process
 - SIGINT (2) - Interrupt signal, typically sent when a user presses Ctrl+C

Signal Handling

- Where should a signal be delivered for multi-threaded?
 - Deliver the signal to the thread to which the signal applies
 - Deliver the signal to every thread in the process
 - Deliver the signal to certain threads in the process
 - Assign a specific thread to receive all signals for the process
 - Examples:
 - `kill (pid_t pid, int signal)`
 - `pthread_kill(pthread_t did, int signal)`

Thread Cancellation

- Terminating a thread before it has finished
- Thread to be canceled is **target thread**
- Two general approaches:
 - Asynchronous cancellation terminates the target thread immediately
 - Deferred cancellation allows the target thread to periodically check if it should be cancelled

Thread-Local Storage

大家都共享資料

- **Thread-Local Storage (TLS)** allows each thread to have its own copy of data
- Different from local variables
 - Local variables visible only during single function invocation
 - TLS visible across function invocations

Scheduler Activations

- One scheme for communication between the user-thread library and the kernel is known as scheduler activation
- The kernel provide an application with a set of **virtual processors (LWPs)**
(類似虛擬處理器, 會自己排程)
- The application can schedule user threads onto an available virtual processor
- Reference: <https://www.thegeekstuff.com/2013/11/linux-process-and-threads/>

